
FIT5196 Task 1 — Complete Interview Study Guide

The Architecture of the Notebook Before You Touch Any Data

Before a single error is fixed or a single null is imputed, the notebook builds five infrastructure components in Section 3. Everything in Sections 4, 5, and 6 depends on these. This ordering is the single most important structural decision in the whole notebook, and the TA can ask about it from multiple angles.

The five components are: meal time windows, per-meal item sets, item unit prices from linear algebra, a Dijkstra-ready road network graph, and per-branch delivery fee models. They are built in dependency order — each one depends on the previous. Item sets require knowing the meal windows. Item prices require the item sets. Fee models require knowing which training source is clean (which itself requires understanding what each file contains). The graph is built independently but the fee model training requires it (distances are already in the training data, but the imputation later needs live distance computation).

Working on a copy (`dirty_fixed = dirty.copy()`) throughout ensures the original dataframes are never mutated. This matters because the original files are used as reference during validation and for the fee model training source.

Section 2: Why These Libraries

`pandas` and `numpy` are standard. Everything else is a deliberate choice.

`networkx` — you need Dijkstra's shortest-path on the road network. NetworkX implements `single_source_dijkstra_path_length` which computes the shortest path from one source node to every reachable node in one pass. That's the key: you run it once per branch and cache the results, rather than running Dijkstra from scratch for each order.

`scipy.spatial.cKDTree` — given a customer's lat/lon coordinates, you need to find the closest node among 17,117 nodes. A brute-force search is $O(n)$ per query. `cKDTree` builds a k-d tree on the node coordinates and answers nearest-neighbour queries in $O(\log n)$. Every distance computation goes through this lookup, so the speed difference is significant across 500 rows.

`ast.literal_eval` — `order_items` is stored as a Python list-of-tuples string like `[('Burger', 2), ('Fries', 1)]`. You can't use `json.loads` here because Python tuples aren't valid JSON syntax. `eval()` would work but executes arbitrary code, which is a security risk. `ast.literal_eval` safely evaluates Python literals only — the correct tool.

`numpy.linalg.lstsq` — solving the item price system. The system is overdetermined (many more equations than unknowns — 151 breakfast orders for 4 unknowns). `lstsq` handles overdetermined systems using least squares. `numpy.linalg.solve` only works on square systems and would fail here.

`sklearn.linear_model.LinearRegression` — the spec mandates this explicitly for the delivery fee model.

`re` — for extracting the alphabetic prefix from `order_id` strings using `re.match(r'([A-Z]+)', order_id)`.

`np.random.seed(42)` — reproducibility. Any stochastic operation produces the same result every run.

Section 3.1: Schema and Null Audit — What the Numbers Tell You

dirty_data: zero nulls in every column, all expected dtypes. The problems here are value errors, not missing values. The mean `customer_lat` of `-30.75` (versus the expected `~-37.8` for Melbourne) and the standard deviation of `25.34` (versus `~0.008` in the other two files) is the first hard signal that coordinates are corrupted. The max `customer_lat` of `145.01` is a physically impossible latitude — latitudes go from `-90` to `+90`. That value is a longitude.

missing_data: 100 nulls in `branch_code` (20%), 50 in `distance_to_customer_KM` (10%), 50 in `delivery_fee` (10%). Zero nulls everywhere else. Coordinates look clean — mean lat around `-37.81`, std `~0.008`.

outlier_data: zero nulls anywhere. But the `delivery_fee` max of `25.73` and the elevated std of `3.16` (versus `~2.5` in dirty and missing) signal that the outliers are already visible in the summary statistics.

branch_code value counts in dirty_data: NS 159, BK 157, TP 154, then tp 11, ns 10, bk 9. The three lowercase variants add up to 30. This is direct confirmation of casing errors before you write a single line of fix code.

Section 3.2: Graphical EDA — What the Plots Confirm

The histogram for `customer_lat` in `dirty_data` shows the main Melbourne cluster compressed to the far left of the x-axis because the anomalous positive-latitude values (and longitude-range values) stretch the scale to the right. This visual alone confirms two coordinate error subtypes exist: missing negative sign and swapped lat/lon.

The coordinate scatter for `dirty_data` shows the Melbourne cluster squeezed into the bottom-right corner of the plot, with anomalous points stretching both axes to ~150. The `missing_data` and `outlier_data` scatters show tight Melbourne clusters with no extremes.

The delivery fee boxplots show similar median fees across all three files (~13–15 per branch). In `dirty_data` and `missing_data`, a small number of low points below the whiskers are visible — these are loyalty-discounted orders at 50% of base fee, not errors. In `outlier_data`, the whisker outliers are more numerous and extend further in both directions, with the upper extreme reaching ~25. This visual separation motivates the IQR approach for Section 6 and confirms the loyalty discount must be reversed before detection.

Section 3.3: Meal Windows and Item Set Derivation

The spec defines:

- Breakfast: 08:00:00 to 12:00:00
- Lunch: 12:00:01 to 16:00:00
- Dinner: 16:00:01 to 20:00:00

`time` is guaranteed error-free. Given any time string, `get_correct_meal` returns the unambiguous meal type. The windows are implemented as `pd.Timedelta` comparisons rather than string parsing.

Item sets are derived programmatically from `outlier_data`. You filter rows by meal window, then collect every item name that appears in orders during that window. The result:

- Breakfast: {Cereal, Coffee, Eggs, Pancake}
- Lunch: {Burger, Chicken, Fries, Salad, Steak}
- Dinner: {Fish&Chips, Pasta, Salmon, Shrimp}

Why `outlier_data`? It is the only file guaranteed free of errors in both `order_items` and `order_price`. Missing data has NaN columns. Dirty data has wrong items and wrong prices. If you derived item sets from dirty data, rows with wrong items would contaminate the set — you'd add invalid items to the reference, which breaks detection entirely.

Why not hardcode the item sets? Because the spec doesn't explicitly list them. You derive them from data so that if the data ever changed, the notebook wouldn't need manual

updates. More importantly for the Q&A: the derivation is self-documenting — you're showing that the item sets are a property of the data, not an assumption you brought in.

The three sets are completely disjoint — no item appears in more than one meal. This is what makes wrong-item detection unambiguous. If you see **Burger** in a row whose time falls in the Breakfast window, that's an error with zero ambiguity.

Section 3.4: Road Network Graph and Dijkstra Pre-computation

`edges.csv` has 42,224 rows. The graph has 25,491 edges. The difference: the raw file stores directed edges — A→B and B→A as separate rows. Building an undirected `nx.Graph()` collapses each pair into one edge. Where duplicate edges exist (same node pair, different distances due to direction), the minimum distance is kept.

Why undirected? The distance from branch to customer is the same as customer to branch. The assignment asks for shortest distance, not directed shortest path. Undirected also means the graph is symmetric, which is what `single_source_dijkstra_path_length` from the branch node gives you — distances from that branch to every other node.

`nx.is_connected(G)` returns `True`. This guarantees every node is reachable from every other node, meaning Dijkstra always finds a path. If the graph were disconnected, some customer nodes would be unreachable and `branch_shortest[bc][customer_node]` would raise a `KeyError`.

Branch node mapping: each branch coordinate in `branches.csv` maps to a node in `nodes.csv` with exact match (difference < 1e-8 degrees). This means branch-to-branch-node distance is zero — the branch sits exactly on a graph node.

Pre-computation: `nx.single_source_dijkstra_path_length(G, bnode, weight='weight')` is run once per branch. Each call computes the shortest path from that branch node to all 17,117 nodes and stores the results in a dictionary. Result: NS, TP, and BK each reach all 17,117 nodes.

Validation: `compute_distance_km` is run against all 500 outlier rows. Max error = 0.00e+00 km — exact match. This proves the graph construction, spatial indexing, and distance computation are all correct before any of them are used.

`compute_distance_km(branch_code, cust_lat, cust_lon):`

1. Calls `find_nearest_node(cust_lat, cust_lon)` — queries the cKDTree to get the node ID of the nearest road intersection to the customer

2. Looks up `branch_shortest[branch_code][customer_node_id]` — dictionary lookup, $O(1)$
3. Divides by 1000 to convert metres to kilometres

The performance design: pre-computation runs Dijkstra 3 times (once per branch). All subsequent distance queries are $O(\log n)$ for the tree lookup plus $O(1)$ for the dictionary lookup. Without pre-computation, you'd run Dijkstra potentially thousands of times.

Section 3.5: Item Unit Price Derivation

The mathematical setup: for a given meal, every order's price satisfies `sum(quantity_i × price_i) = order_price`. Write this out for all orders of that meal and you get a linear system `Ax = b` where:

- Each row of `A` is an order, each column is a menu item, entry `A[i, j]` is the quantity of item `j` in order `i`
- `b` is the vector of order prices
- `x` is the vector of unknown unit prices

For Breakfast: 151 equations, 4 unknowns. For Lunch: 189 equations, 5 unknowns. For Dinner: 160 equations, 4 unknowns. All heavily overdetermined.

`numpy.linalg.lstsq(A, b, rcond=None)` solves this. The fourth return value is rank. The key check is the max residual `|b - A @ prices|.max()`:

- Breakfast: rank=4, max_residual=2.27e-13
- Lunch: rank=5, max_residual=4.55e-13
- Dinner: rank=4, max_residual=3.41e-13

These residuals are machine-precision zero — confirming the least-squares solution is also the exact solution, which means the source data has no errors in either `order_items` or `order_price` for `outlier_data`.

Rank = number of unknowns in all three cases. This means the system is fully determined — there is a unique solution. If rank were less than the number of unknowns, the system would be underdetermined and you'd have multiple valid price vectors.

Recovered prices:

Item	Price	Item	Price	Item	Price

Cereal	\$21.00	Burger	\$31.00	Fish&Chips	\$35.00
Coffee	\$7.50	Chicken	\$32.00	Pasta	\$27.50
Eggs	\$22.00	Fries	\$12.00	Salmon	\$41.00
Pancake	\$24.25	Salad	\$17.20	Shrimp	\$54.00
		Steak	\$45.00		

Why not read prices from a lookup table? No such table exists in the spec. You derive them from data. This also means the prices are validated against the data — they're not assumptions.

Why can't you use dirty_data? Dirty data has rows with wrong items and wrong prices. A single row with a wrong item inserts an incorrect equation into the system, corrupting the solution. The residuals would not be near zero.

Section 3.6: Delivery Fee Model — Training Source Decision

Training source: `missing_data` rows where `dropna()` leaves no nulls.

This is the most defensible choice and the TA will probe it. The logic:

`dirty_data` — has value errors. Wrong distances, wrong loyalty flags, wrong prices. Any corrupted feature value in a training row produces incorrect model coefficients. The model would learn from noise.

`outlier_data` — has outlier delivery fees. Training the fee model on outlier fees would teach the model that outlier fees are normal. The model's baseline would be skewed, which then corrupts downstream outlier detection and loyalty flag validation.

`missing_data` with `dropna()` — the spec guarantees `missing_data` contains only coverage gaps, no value errors. Rows that are complete (no nulls) are therefore error-free. 350 clean rows remain after dropping nulls (116 NS + 114 TP + 120 BK).

Loyalty discount reversal before training: Customers with `customerHasloyalty? = 1` pay 50% of the base fee. If you train on raw `delivery_fee`, you mix two populations: non-loyalty customers paying full fee, loyalty customers paying half. A linear model trying to fit both produces a worse fit and lower R^2 . `get_base_fee` reverses this: if `customerHasloyalty? == 1`, return `delivery_fee / (1 - 0.50) = delivery_fee / 0.50`. The model then learns the base fee exclusively. Post-prediction, the loyalty discount is reapplied.

Model coefficients:

Branch NS (n=116, $R^2=0.9581$):

- Intercept: 4.6601
- weekend: +2.1330 per weekend day vs weekday
- time_of_day: +0.5087 per step (morning→afternoon→evening)
- distance: +1.0321 per km

Branch TP (n=114, $R^2=0.9602$):

- Intercept: 3.8366
- weekend: +1.4590
- time_of_day: +0.6330
- distance: +0.8749

Branch BK (n=120, $R^2=0.9829$):

- Intercept: 4.5184
- weekend: +2.4339
- time_of_day: +0.9872
- distance: +1.0526

All three R^2 values exceed the 0.95 threshold from the spec. BK has the highest R^2 (0.9829) and also the highest sensitivity to weekend and time-of-day. TP has the smallest distance coefficient (0.87) and weekend premium (1.46).

The high R^2 values serve a dual purpose: they confirm the models are well-fitted, and they confirm the training data is clean. A corrupted training set would have produced lower R^2 because errors in features or target values add unexplained variance.

Why `LinearRegression` and not Ridge, Lasso, or anything else? The spec mandates `sklearn.linear_model.LinearRegression` explicitly. There's no ambiguity.

Why continuous features? The spec says "treat all variables as continuous — no one-hot encoding needed." `weekend_flag` (0 or 1) and `time_of_day` (0, 1, or 2) are fed in as-is. This is unusual (weekends are technically categorical) but the spec is explicit.

Section 3.7: predict_fee and predict_fee_raw

Two wrapper functions exist for the fee model:

`predict_fee(branch_code, date_str, time_str, distance_km, has_loyalty)` — takes a row's fields and returns the predicted fee (with loyalty discount applied if applicable). Used in Section 4.8 for dirty data loyalty flag detection, where you call it row by row via `apply`.

`predict_fee_raw(branch_code, date_str, time_str, distance_km, has_loyalty)` — functionally identical but takes all parameters explicitly rather than deriving them from a row object. Used in Section 5.4 for missing data fee imputation, where you're constructing inputs programmatically rather than pulling from a row.

Both internally: derive `weekend` via `get_weekend_flag(date_str)`, derive `time_of_day` via `get_time_of_day(time_str)`, construct `X = np.array([[weekend, time_of_day, distance_km]])`, call `fee_models[branch_code].predict(X)[0]` to get the base fee, then multiply by 0.5 if loyalty applies.

Section 4: Dirty Data — The Fix Order Is Not Arbitrary

Fixes happen in dependency order: structural errors first, model-dependent errors last.

Date → Branch Code → Coordinates → Order Type → Items → Price → Distance → Loyalty Flag

The reason: each fix category either enables or relies on the previous one.

- You fix dates first because they affect `get_weekend_flag`, which is used in fee prediction later.
- You fix branch codes before coordinates because distance-matching (used to fix wrong branches) needs correct coordinates to compute Dijkstra distances.
- Wait — actually coordinates are fixed *after* branch codes in the notebook, which creates a subtle dependency. The branch code fix uses `distance_to_customer_KM` (guaranteed error-free) and current coordinates. For the 13 rows where the branch itself is wrong, the coordinates at that point haven't been fixed yet. This works because those 13 rows have a branch code error but not a coordinate error (one anomaly per row), so their coordinates are already correct for distance-matching.
- Order type is fixed before items because item validation depends on `order_type`.
- Items are fixed before prices because price validation depends on items being correct.

- Distance is fixed after coordinates and branch codes because both affect the Dijkstra distance.
- Loyalty is fixed last because it requires the fee model, which needs correct distances and branch codes.

`fix_log = {}` tracks fix counts per anomaly type. This is how the final summary (299 total) is produced.

4.1 Date Format Errors — 37 Fixed

Detection: `pd.to_datetime(dates, errors='coerce')` — invalid dates return NaT. Counting NaT gives the error count.

Two malformed patterns:

- `YYYY-DD-MM` — e.g., `2018-25-11` (year at front, day and month swapped)
- `DD-MM-YYYY` — e.g., `01-09-2018` (year at end)

Why are these unambiguous? The day value always exceeds 12 in the malformed rows. No month 25, 27, or 23 exists, so you can always determine which component is the day. A case like `2018-09-11` would be genuinely ambiguous (September 11 or November 9), but that doesn't occur here.

The fix function `fix_date`: tries three format strings in order. `%Y-%m-%d` first (valid dates return unchanged), then `%Y-%d-%m`, then `%d-%m-%Y`. The first match wins. All output dates are returned as `YYYY-MM-DD` strings via `.strftime('%Y-%m-%d')`.

Validation: `pd.to_datetime(dirty_fixed['date'], errors='coerce').isna().sum() == 0`.

4.2 Branch Code Casing Errors — 30 Fixed

Detection: any `branch_code` not in `{ 'NS', 'TP', 'BK' }`. Found: tp (11), ns (10), bk (9) = 30 rows.

The naive fix would be `.upper()`. This is insufficient.

The diagnostic: for each of the 30 lowercase rows, compare the branch from naive uppcasing against the branch whose Dijkstra distance best matches the recorded `distance_to_customer_KM`. Result: **13 of the 30 rows have a branch value that is wrong beyond just casing**. Naive uppcasing assigns the wrong branch to these 13 rows.

Why does this work? `distance_to_customer_KM` is guaranteed error-free for these rows. Each row has at most one anomaly — if the anomaly is the branch code, then the

distance is correct. The Dijkstra distance from a specific branch to a specific customer node is unique. So the branch whose computed distance matches the recorded distance is, by definition, the correct branch.

The fix: for all 30 rows, iterate over `{'NS', 'TP', 'BK'}`, compute the Dijkstra distance from each branch to the customer, keep the branch with minimum absolute error. For pure casing errors, this returns the same result as uppercasing. For the 13 wrong-branch rows, it returns the correct branch.

Validation: `dirty_fixed['branch_code'].isin(valid_branches).all()`.

4.3 Customer Coordinate Errors — 41 Fixed

Sub-pattern 1 — Missing negative sign (37 rows): `customer_lat` is positive (e.g., 37.8146) but should be negative (−37.8146). Melbourne is in the Southern Hemisphere. `customer_lon` is already in the valid longitude range (`lon_min` to `lon_max` from `nodes.csv`).

Detection mask: `(customer_lat > 0) & (customer_lat < 90) & (customer_lon.between(lon_min, lon_max))`. Fix: `dirty_fixed.loc[mask, 'customer_lat'] = -dirty_fixed.loc[mask, 'customer_lat']`.

Sub-pattern 2 — Fully swapped lat/lon (4 rows): `customer_lat` holds a longitude value (~144–145), `customer_lon` holds a negative latitude (~−37.8).

Detection mask: `(customer_lat.between(lon_min, lon_max)) & (customer_lon < 0)`. Fix: snapshot both columns before touching either:

```
snap_lat = dirty_fixed.loc[swapped_idx, 'customer_lat'].values.copy()
```

```
snap_lon = dirty_fixed.loc[swapped_idx, 'customer_lon'].values.copy()
```

```
dirty_fixed.loc[swapped_idx, 'customer_lat'] = snap_lon
```

```
dirty_fixed.loc[swapped_idx, 'customer_lon'] = snap_lat
```

Why the snapshot? Without it: `lat = lon` overwrites `lat`. Then `lon = lat` copies the already-overwritten value. Both columns end up holding the same number. The `.copy()` preserves both original values before either is changed.

Why are the two patterns distinguishable? They produce distinct numeric ranges. Missing negative sign → `customer_lat` is in 37–38 (positive latitude range). Swap → `customer_lat` is in 144–145 (longitude range). The masks are mutually exclusive.

Validation: all latitudes within `[lat_min - 0.01, lat_max + 0.01]`, all longitudes within `[lon_min - 0.01, lon_max + 0.01]` from the node network bounds.

Total: $37 + 4 = 41$ coordinate fixes.

4.4 Order Type Errors — 37 Fixed

`time` is guaranteed error-free. `get_correct_meal(time_str)` maps each time to its correct meal window. Add as a helper column and compare against `order_type`. Any mismatch → wrong `order_type`.

Fix: overwrite `order_type` with `correct_meal`. The helper column `correct_meal` is dropped before writing the solution — it's not in the output schema.

Validation: `(dirty_fixed['order_type'] == dirty_fixed['correct_meal']).all()`.

This section is simple but important as a dependency step. Items cannot be validated until `order_type` is correct.

4.5 Wrong Menu Item — 37 Fixed

With `order_type` correct, parse each `order_items` string with `ast.literal_eval` and check every item name against `item_sets[order_type]`. Any item not in the set is wrong.

The residual price method:

1. Separate the items in the order into valid (belong to the meal) and wrong (don't belong)
2. Compute `accounted = sum(item_prices[name] * qty for name, qty in parsed if name in valid)`
3. `residual = order_price - accounted` — this is the price explained by the wrong item
4. `implied_price = residual / wrong_qty` — the unit price the wrong item must have
5. Find which valid menu item has `abs(item_prices[candidate] - implied_price) < 0.01`

Why does this work? `order_price` is guaranteed error-free. Item quantities inside `order_items` are guaranteed error-free. So `residual` is exactly `wrong_qty ×`

`correct_unit_price`. Dividing by `wrong_qty` recovers the unit price, and since all item prices are unique within a meal, this uniquely identifies the replacement.

Why can item prices be guaranteed unique within a meal? Because the linear system in Section 3.5 has a unique solution and each recovered price is numerically distinct from all others in the same meal.

Validation: check that no row has any item outside its meal's valid set after the fix.
`remaining_wrong == 0`.

The helper column `correct_meal` added here is dropped before the next section.

4.6 Wrong Order Price — 37 Fixed

With `order_items` now clean, the correct price for any row is `sum(quantity × unit_price)`. `compute_order_price(order_items_str)` does this. Add as `expected_price` helper column. Flag rows where `|order_price - expected_price| > 0.01`.

Fix: overwrite with `expected_price`. Drop the helper column.

Why 0.01 as the threshold? Floating-point arithmetic can produce tiny rounding errors even when the underlying value is correct. \$0.01 is well above floating-point noise but well below the smallest price difference between any two menu items.

Why 37? These are rows where `order_price` was the primary error — not `order_items`. In Section 4.5, the items were already correct (those were wrong-item rows). Here, the items were already correct to begin with and the price field itself was wrong. The one-anomaly-per-row rule means these never overlap with Section 4.5.

Validation: `(dirty_fixed['order_price'] - dirty_fixed['expected_price']).abs() > 0.01).sum() == 0`.

4.7 Wrong Distance — 43 Fixed

With branch codes and coordinates now correct, recompute `distance_to_customer_KM` for every row using `compute_distance_km`. Flag rows where recorded vs computed differs by `> 0.01` km. Fix: overwrite with computed value.

Why 43 when most other types are 37? The cascade effect. When coordinates were fixed in Section 4.3 (41 rows), the customer's nearest node in the graph changed, which changed their Dijkstra distance. These rows require a distance update even though the distance itself was not the primary error — the coordinate was. The 43 includes:

- Rows with a standalone wrong distance value
- Rows from Section 4.3 whose corrected coordinates change their nearest node and thus their Dijkstra distance

There's also a contribution from the 13 rows in Section 4.2 where the branch itself was wrong — fixing the branch changes which branch Dijkstra runs from, changing the distance. These all cascade into the distance fix count.

Validation: `(dirty_fixed['distance_to_customer_KM'] - dirty_fixed['computed_dist']).abs() > 0.01).sum() == 0.`

4.8 Wrong Loyalty Flag — 37 Fixed

With distances correct, validate `delivery_fee` against the fee model. For each row, compute:

- `err_current` = `|recorded_fee - predict_fee(..., has_loyalty=recorded_loyalty)|`
- `err_flipped` = `|recorded_fee - predict_fee(..., has_loyalty=1 - recorded_loyalty)|`

Flag condition: `err_current > 2.0 AND err_flipped < 1.5.`

Fix: `dirty_fixed.loc[mask, 'customerHasloyalty?'] = 1 - dirty_fixed.loc[mask, 'customerHasloyalty?'].`

Why dual thresholds? The fee model has residual error ($R^2 \sim 0.96$, not 1.0). Normal rows have small prediction errors under the correct loyalty assumption — not necessarily zero. A single threshold like "flag if `current_error > 2.0`" would also flag rows with genuine model residual. The dual condition — current error large AND flipped error small — only fires when the fee fits badly under the recorded loyalty AND fits well under the opposite loyalty. A normal row with model residual would have moderate error under both assumptions.

Why is `delivery_fee` the reliable signal here? Because the spec guarantees `delivery_fee` is error-free in dirty data. So if the recorded fee doesn't fit the model with the recorded loyalty flag, the loyalty flag must be wrong — the fee isn't.

Validation after fix: check that no row is still better explained by the flipped loyalty assumption — specifically, that no row has `err_final_flipped < err_final - 2.0`. All such checks pass after the 37 fixes.

All four helper columns (`err_current`, `err_flipped`, `err_final`, `err_final_flipped`) are dropped before writing.

4.9 Validation and Summary

Eight independent checks, one per anomaly type, all using the same detection logic as the corresponding fix section but applied to the final state of `dirty_fixed`. All pass.

Fix summary confirmed from notebook output:

Anomaly type	Rows fixed
date_format	37
branch_code_casing	30
customer_coordinates	41
order_type	37
wrong_menu_item	37
order_price	37
distance	43
loyalty_flag	37
Total	299

"299 corrections but only 500 rows — does that mean some rows have multiple fixes?" No. One anomaly per row. But fixing the primary anomaly triggers required updates to dependent fields. The most common cascade: coordinate fix → distance update. The 299 count is column-level corrections, not the number of anomalous rows. The number of rows with a genuine primary error is lower.

Schema check before writing: `assert list(dirty_fixed.columns) == list(dirty.columns)`. Column names must be identical to the input. The auto-marker scores zero if they differ.

Output shape confirmed: (500, 12). Read-back check passed.

Section 5: Missing Data — Three Columns, Three Distinct Methods

5.1 Null Structure and Dependency Order

Null counts: `branch_code` 100, `distance_to_customer_KM` 50, `delivery_fee` 50.

Overlap analysis output from notebook:

- Branch null only: 50
- Branch + distance null: 50
- Distance null only: 0
- Fee null only: 50
- All three null: 0

"Distance null only: 0" is the critical finding. Every row with a null distance also has a null branch. You cannot compute Dijkstra distance without knowing which branch to measure from. So the imputation order is forced: `branch_code` first, then `distance_to_customer_KM`, then `delivery_fee`.

The 50 fee nulls are completely independent ("All three null: 0") — they can be done in any order relative to each other, but only after branch and distance are complete because the fee model needs both.

5.2 Branch Code Imputation — 100 Rows

The 100 null rows split into two groups by the null structure: Group 1 has null branch but known distance (50 rows). Group 2 has both null (50 rows).

Methods evaluated — Group 1 (distance known):

Distance matching: for a given customer location, compute Dijkstra distance from each of the three branches. The branch whose computed distance most closely matches the recorded `distance_to_customer_KM` is the correct branch. Validated on 400 known rows: **100% accuracy (400/400)**. This is exact because Dijkstra distance uniquely identifies the branch — no two branches produce the same distance to the same customer node.

Applied to Group 1.

Methods evaluated — Group 2 (both null):

Mode imputation: assigns the most frequent branch to all null rows. Ignores geography entirely. Correct only by chance for at most one-third of rows. Rejected immediately.

KNN imputation: probabilistic estimate from feature similarity. Could work with customer coordinates as features, but gives an approximate answer when an exact answer is available. Rejected.

Fee-based matching (Method 2): for each branch, compute Dijkstra distance from that branch to the customer, predict the fee using the linear model, pick the branch with minimum prediction error against the recorded fee. Validated on 400 known rows: **74% accuracy (296/400)**. The 26% failure rate occurs because the fee model has residual error — two branches can produce similar predicted fees for the same customer location, causing misclassification. Insufficient.

Order ID prefix mapping: each `order_id` has an alphabetic prefix (ORDA, ORDB, etc.) that is a deterministic branch identifier. The mapping is derived from `outlier_data` (clean, all 500 rows have known branch codes) using `re.match(r'([A-Z]+)', order_id)`. An assertion confirms each prefix maps to exactly one branch — self-validating. Validated on 400 known rows in `missing_data` and all 500 rows in `outlier_data`: **100% accuracy in both datasets (900/900 total)**.

The derived mapping from the notebook:

Prefix	Branch
ORDA	BK
ORDB	TP
ORDC	NS
ORDI	NS
ORDJ	TP

ORDK	BK
ORDX	BK
ORDY	TP
ORDZ	NS

Applied to Group 2.

Post-imputation branch distribution: BK 172, TP 165, NS 163. Consistent with the original non-null row proportions.

Validation of branch imputation (indirect): the distance consistency check in Section 5.5 re-computes Dijkstra distance from the imputed branch to the customer for every row. If any branch were imputed wrong, the computed distance would differ from the recorded distance and the assertion would fail.

5.3 Distance Imputation — 50 Rows

With `branch_code` fully imputed, call `compute_distance_km(branch_code, customer_lat, customer_lon)` for each of the 50 rows still missing distance. This is exact computation, not estimation.

Imputed distance stats from notebook:

- count: 50, mean: 8.9602 km, std: 1.5256 km, min: 3.8270 km, max: 11.8060 km

Consistent with the non-null distance distribution in the file (mean ~8.60, std ~1.68). No implausible values. Each value is the unique Dijkstra shortest-path distance for that branch-customer node pair.

Why not use mean imputation, KNN, or regression for distance? Because you have all the information to compute the exact value: the corrected branch code and the customer coordinates. The business rule defines a unique correct value. Any statistical method would produce an approximation of something you can compute exactly. That's not a trade-off worth making.

5.4 Delivery Fee Imputation — 50 Rows

All inputs required by `predict_fee_raw` are now complete for all 50 null rows: branch code, date, time, distance, loyalty flag.

Call `predict_fee_raw(branch_code, date, time, distance_km, customerHasloyalty?)` for each row.

Imputed fee stats from notebook:

- count: 50, mean: 13.6437, std: 2.3927, min: 6.1507, max: 18.4861

The minimum of \$6.15 corresponds to a loyalty-discounted order — approximately half of the minimum expected base fee. This is expected, not anomalous.

Consistent with the non-null fee distribution in the file (overall mean ~13.88).

Why model-based rather than mean/KNN/regression imputation? Same logic as distance: the business rule gives you the exact value. The linear model is deterministic given its inputs. Statistical imputation would be a worse approximation of something you can compute exactly.

5.5 Validation

Four checks:

1. `missing_fixed.isnull().sum().sum() == 0` — no nulls anywhere
2. `missing_fixed.shape == (500, 12)` — shape unchanged
3. `missing_fixed['branch_code'].isin(valid_branches).all()` — all branch codes valid
4. All distances match Dijkstra: `(|distance_to_customer_KM - computed_dist| > 0.01).sum() == 0` — this also implicitly validates branch imputation

All four pass. Solution written, read-back confirmed.

Section 6: Outlier Detection and Removal

6.1 Why You Can't Apply IQR Directly to the Raw Fee Column

Loyalty customers pay 50% of the base fee. In the raw `delivery_fee` column of `outlier_data`:

- Loyalty=1: n=24, mean fee=\$6.49
- Loyalty=0: n=476, mean fee=\$14.25
- Ratio: 2.195 (expected ~2.0)

The slight deviation from exactly 2.0 is because outlier fees exist in the non-loyalty group, pulling their mean up.

If you apply IQR to the raw fees, loyalty customer fees cluster at \$6–8 — well below $Q1 - 1.5 \times IQR$ for the combined distribution — and get incorrectly flagged as outliers. They're not outliers. They're discounted fees.

Fix: `get_base_fee(row)` reverses the discount:

- If `customerHasloyalty? == 1`: return `delivery_fee / (1 - 0.50) = delivery_fee * 2`
- Otherwise: return `delivery_fee` unchanged

Base fee stats after reversal:

- mean: 14.1897, std: 2.7452, min: 5.6049, max: 25.7305

Apply IQR to `base_fee`. Flag outlier rows. Remove those rows from the original `outlier_fixed` dataframe (not the `base_fee` version). Drop `base_fee` before writing.

6.2 IQR Method — Why and How

Why IQR over z-score?

Z-score: $(\text{value} - \text{mean}) / \text{std}$. The mean and standard deviation are sensitive to the values you're trying to detect. A few extreme outliers pull the mean up and inflate the std, which compresses the z-scores of the outliers and makes them look less extreme than they are. The method is self-defeating when the distribution has heavy tails.

IQR is quartile-based. $Q1$ and $Q3$ are by definition not influenced by values below $Q1$ or above $Q3$. The fence boundaries — $Q1 - 1.5 \times IQR$ and $Q3 + 1.5 \times IQR$ — are robust to extreme values. The 1.5 multiplier is the standard Tukey fence for univariate outlier detection.

Why per-branch, not across all orders combined?

Each branch has a different fee model with different intercepts and coefficients. Branch NS: intercept 4.66, distance coefficient 1.03. Branch TP: intercept 3.84, distance 0.87. Branch BK: intercept 4.52, distance 1.05. Their base fee distributions are not directly comparable — BK orders will tend to have higher fees than TP orders simply because BK has larger coefficients, not because TP has outliers. Pooling the three distributions and applying a single IQR threshold conflates normal branch variation with genuine outliers.

Results per branch:

Branch	Q1	Q3	IQR	Lower bound	Upper bound	Outliers
NS	13.358	15.769	2.411	9.741	19.386	12
TP	12.317	14.454	2.137	9.112	17.660	13
BK	12.683	15.924	3.241	7.822	20.785	13

Total: 38 rows flagged. $500 - 38 = 462$ rows remaining. Index reset to 0–461.

BK has the widest IQR (3.241) because its fee distribution is more spread — a higher distance coefficient and weekend premium means greater natural fee variance.

6.3 Why 4 New Apparent Outliers After Removal Are Not a Problem

After removing 38 rows, re-running IQR on the 462-row cleaned dataset shows: NS 0, TP 3, BK 1 rows outside the new bounds.

This is expected and correct. Removing the original outliers changes Q1, Q3, and therefore IQR. The new quartiles are based on a tighter distribution (the extreme values are gone). This shifts Q1 and Q3 inward, which reduces IQR, which tightens both fence boundaries. Values that were inside the original bounds but near the edge can fall outside the recalculated bounds.

You do not iterate. The detection criterion is IQR computed on the original 500-row dataset with the original fence values. Applying IQR again to the cleaned dataset is a different statistical test with different parameters. Iterating would progressively trim valid data that sits at the natural tail of the distribution, not genuine outliers. The 38 removals correctly represent the outliers under the original distribution. No further removal is warranted.

Numbers to Have Ready — All Confirmed From Notebook Output

Fix counts (dirty data):

Type	Count
Date format	37
Branch code casing	30
Customer coordinates	41 (37 missing negative, 4 swapped)
Order type	37
Wrong menu item	37
Wrong order price	37
Wrong distance	43
Wrong loyalty flag	37
Total operations	299

Missing data:

Column	Imputed	Method	Accuracy
branch_code (Group 1)	50	Dijkstra distance matching	100%

branch_code (Group 2)	50	order_id prefix mapping	100%
distance_to_customer_KM	50	Dijkstra exact computation	Exact
delivery_fee	50	Per-branch linear model	Exact

Fee matching accuracy (rejected): 74% (296/400)

Fee model:

Branch	n	R ²	Intercept	weekend	time_of_day	distance
NS	116	0.9581	4.6601	+2.1330	+0.5087	+1.0321
TP	114	0.9602	3.8366	+1.4590	+0.6330	+0.8749
BK	120	0.9829	4.5184	+2.4339	+0.9872	+1.0526

IQR bounds:

Branch	Q1	Q3	IQR	Lower	Upper	Outliers removed
NS	13.358	15.769	2.411	9.741	19.386	12
TP	12.317	14.454	2.137	9.112	17.660	13
BK	12.683	15.924	3.241	7.822	20.785	13

Total						38
--------------	--	--	--	--	--	-----------

Rows remaining after outlier removal: **462**

Item prices:

Meal	Item	Price
Breakfast	Coffee	\$7.50
Breakfast	Cereal	\$21.00
Breakfast	Eggs	\$22.00
Breakfast	Pancake	\$24.25
Lunch	Fries	\$12.00
Lunch	Salad	\$17.20
Lunch	Burger	\$31.00
Lunch	Chicken	\$32.00
Lunch	Steak	\$45.00
Dinner	Pasta	\$27.50

Dinner	Fish&Chips	\$35.00
Dinner	Salmon	\$41.00
Dinner	Shrimp	\$54.00

Prefix → Branch mapping: ORDA→BK, ORDB→TP, ORDC→NS, ORDI→NS, ORDJ→TP, ORDK→BK, ORDX→BK, ORDY→TP, ORDZ→NS

Every Q&A Angle the TA Can Take

"Why did you pre-compute Dijkstra instead of computing it per row?" Running `single_source_dijkstra_path_length` once per branch gives you the shortest path from that branch to every node in one pass. Subsequent lookups are $O(1)$ dictionary hits. Computing Dijkstra from scratch per order would mean hundreds of Dijkstra runs on a 17,117-node graph.

"Why undirected graph?" Distance from A to B equals distance from B to A. We want shortest distance, not directed shortest path. Undirected also halves the edge count (42,224 directed → 25,491 undirected).

"Why cKDTree for node lookup?" 17,117 nodes. Brute-force nearest neighbour is $O(n)$ per query. cKDTree is $O(\log n)$. Every distance computation goes through this.

"Why use outlier_data to derive item sets and prices?" It's the only file guaranteed error-free in both `order_items` and `order_price`. Missing data has NaN columns. Dirty data has wrong items and wrong prices — using it would corrupt both the item sets and the linear system.

"Why train the fee model on missing_data not dirty_data?" Dirty data has value errors including wrong distances and wrong loyalty flags. Wrong feature values corrupt model coefficients. Missing data's only problem is NaN gaps — complete rows are error-free. All three R^2 values exceed 0.95, confirming the training data is clean.

"Why not train on outlier_data?" It contains outlier delivery fees. Training on them teaches the model that outlier fees are normal, corrupting the model's baseline and everything downstream.

"Why reverse the loyalty discount before training?" Loyalty customers pay 50% of base fee. Mixing them with non-loyalty customers in training creates two populations the linear

model tries to fit simultaneously, reducing R^2 . Reversing gives one population — base fees — which the model fits cleanly.

"Why not just uppercase the branch codes?" 13 of the 30 lowercase rows have a branch value that is wrong beyond casing — the recorded distance matches a different branch entirely. Simple uppercasing would assign the wrong branch to those 13 rows. Distance matching is required.

"Why does the distance fix count (43) exceed the other fix counts (37)?" Cascade effect. Fixing coordinates (41 rows in 4.3) changes the customer's nearest node, which changes their Dijkstra distance. These rows require distance updates even though distance wasn't their primary error. Fixing wrong branch codes (13 rows in 4.2) also changes which branch Dijkstra runs from, cascading into distance.

"Why the snapshot-before-swap for coordinate swaps?" Without it, the first assignment overwrites one column, and the second assignment copies the already-overwritten value. Both columns end up holding the same number. The `.copy()` preserves both originals before either is written.

"How did you identify the two coordinate error subtypes?" From EDA. `Dirty_data`'s lat distribution and scatter plot showed values both in the range 37–38 (positive, missing negative sign) and in the range 144–145 (longitude range, indicating a full swap). The two masks are geometrically distinct and mutually exclusive.

"Why IQR over z-score for outlier detection?" Z-score's mean and std are sensitive to the outliers themselves — extreme values inflate both, which compresses the z-scores of outliers and makes them look less extreme. IQR is quartile-based and unaffected by extreme values, making it robust for detecting what it's supposed to detect.

"Why per-branch IQR?" Each branch has different fee model coefficients — BK has a higher distance coefficient and weekend premium than TP. Their base fee distributions are not comparable on the same scale. Pooling them conflates natural branch variation with genuine outliers.

"Why reverse loyalty discount before outlier detection?" Without it, loyalty customers' ~half-price fees cluster at the low end and fall below $Q1 - 1.5 \times IQR$, getting incorrectly flagged as outliers. They're discounted fees, not anomalous ones.

"Why do 4 new outliers appear after removal?" Removing the original 38 tightens $Q1$, $Q3$, and IQR . The new fence boundaries are closer to the center, and values near the original edge can fall outside the recalculated bounds. You don't iterate because the criterion is the IQR of the original distribution. Iterating is a different test and would progressively trim valid data.

"Why is the branch imputation self-validating in the distance check (Section 5.5)?" A wrong branch produces a different Dijkstra distance. The distance consistency check recomputes Dijkstra from the imputed branch for every row and asserts the computed distance matches the recorded value within 0.01 km. If any branch were imputed wrong, this check fails. It passes — confirming branch imputation was correct.

"Could you have used fee matching for Group 2 branch imputation?" No. It achieved only 74% accuracy on known rows, meaning 26% of imputed branches would be wrong. The prefix mapping achieves 100% accuracy validated on 900 known rows across two files and is deterministic. There's no reason to use an inferior method.

"What does the rank output from lstsq tell you?" Rank equals the number of non-redundant equations. Rank = number of unknowns means the system is fully determined — unique solution. Rank < unknowns would mean the system is underdetermined — infinitely many solutions, no unique prices.

"What do machine-precision residuals (2.27e-13) tell you about the source data?" That the source data (outlier_data) has no errors in `order_items` or `order_price`. If even one row had a wrong item or price, the least-squares solution would not be the exact solution, and residuals would be non-negligible. Near-zero residuals confirm the file is genuinely clean for those columns.

"Why is `delivery_fee` guaranteed error-free in dirty_data but other columns aren't?" The spec explicitly states it. The spec also explicitly states which columns can contain errors. For dirty data: `order_id`, `time`, item quantities inside `order_items`, and `delivery_fee` are all guaranteed error-free. This is why `delivery_fee` is the signal used to detect wrong loyalty flags — you know the fee is correct, so if it doesn't fit the model with the recorded loyalty, the flag must be wrong.